

Java – Spring Boot

Seminar-Skript

Dependency Injection · Spring Boot · REST · Spring Data JPA · AOP

Jo Wagner
September 2026

Inhalt

Einleitung: Warum Dependency Injection?	4
Dependency Injection von Hand: Das CalculatorProject	5
2.1 Das Problem: feste Kopplung	5
2.2 Gegen eine Abstraktion programmieren	5
2.3 Konstruktor-Injektion	5
2.4 Dekorierer: Abhängigkeiten kombinieren	6
2.5 Ausblick: der dynamische Proxy	6
2.6 Manuelle Verdrahtung in main()	7
2.7 Zusammenfassung	7
Spring-Grundlagen: Beans, Injektion, Lifecycle	8
3.1 Vom manuellen main() zum Spring-Container	8
3.2 Was ist eine Bean? – @Component	8
3.3 Injektionsarten: Konstruktor, Setter/Field, und ihre Reihenfolge	8
3.4 Injektion mit Lombok: @RequiredArgsConstructor	9
3.5 @Value: Werte statt Objekte injizieren	9
3.6 Bean-Lifecycle: @PostConstruct und @PreDestroy	9
3.7 Singleton, Prototype und Web-Scopes	10
3.8 Wenn mehrere Kandidaten passen: Konflikte auflösen	10
3.9 Vom manuellen Main.java zur automatischen Verdrahtung	10
3.10 Zusammenfassung	11
REST-Endpoints: der Presentation Layer	12
4.1 Ressourcen-Naming	12
4.2 Minimalbeispiel: DemoController	12
4.3 Vollständiger Endpoint: PersonenController	12
4.4 PersonDto	13
Persistence Layer mit Spring Data JPA	15
5.1 Vom Java-Objekt zur Datenbanktabelle: @Entity	15
5.2 Repository: Interface statt Implementierung	15
5.3 Abgeleitete Query-Methoden	15
5.4 Zusammenfassung	16
Domain Layer und Mapping	17
6.1 Person als fachliches Modell	17
6.2 Mapper: MapStruct statt Hand-Code	17
Der Service-/Business-Layer	18
7.1 Vertrag und fachliche Prüfung	18

7.2 Die Transaktionsklammer: @Transactional	18
7.3 @Configuration und @Bean	19
7.4 @ConfigurationProperties: komplexe Werte aus externen Quellen	19
7.5 Ein Seitenblick auf SOLID	20
7.6 Unit-Testing mit Mockito	20
Fehlerbehandlung	21
8.1 Der zentrale ErrorHandler: @ControllerAdvice	21
8.2 Fachliche Exceptions auf Statuscodes abbilden	21
Größere Übung: alle Schichten selbst bauen (Schwein)	23
9.1 Aufgabenstellung	23
9.2 In welcher Reihenfolge bauen?	23
9.3 Musterlösung	23
Integrationstests gegen den laufenden Container	26
10.1 Vom Unit-Test zum echten HTTP-Test	26
Lose Kopplung durch Events	27
11.1 ApplicationEventPublisher	27
Fortgeschrittene Repository-Strategien	28
12.1 @Query, Projektionen und Custom-Repositories	28
Aspektororientierte Programmierung (AOP) mit Spring	30
13.1 @Aspect und @Pointcut	30
13.2 Before-/After-Advice	30
13.3 @Around: volle Kontrolle	31
13.4 Eigene Annotation als Pointcut	31
13.5 Vergleich zu Kapitel 2	31
Ausblick: was noch fehlt	33

1 · Tag 1

Einleitung: Warum Dependency Injection?

Spring ist in erster Linie ein **Dependency-Injection-Container** (kurz: DI-Container, auch IoC-Container genannt – Inversion of Control). Bevor wir Spring anfassen, wollen wir verstehen, welches Problem Spring eigentlich löst: DI ist kein Spring-Feature, sondern ein Entwurfsprinzip. Spring ist nur ein Werkzeug, das dieses Prinzip automatisiert.

Ein nicht-triviales Programmsystem besteht aus vielen Objekten, die kooperieren. Ein Objekt, das auf ein anderes angewiesen ist, sollte nicht direkt von dessen konkreter Klasse abhängen – Dienste sollten über Interfaces angeboten werden. Dann sind sie austauschbar, lassen sich mocken und mit Proxies umgeben.

Merksatz: „Eine Klasse soll ihre Abhängigkeiten bekommen, nicht sich selbst besorgen.“
(Hollywood-Prinzip: „Don't call us, we'll call you.“*)

Ein DI-Container muss wissen, welche Objekte erzeugt und wie sie verbunden werden müssen. Früher hat man diese Metadaten in XML-Dateien beschrieben – dieses Skript verwendet ausschließlich die moderne, annotationsbasierte Variante, wie sie in jedem aktuellen Spring-Boot-Projekt Standard ist.

Das Skript ist auf drei Seminartage aufgeteilt und wird Schritt für Schritt aufgebaut – jedes Kapitel gehört zu einem Beispielprojekt im Kurs-Repository:

- **Tag 1:** CalculatorProject (DI ohne Spring) &rrar; SpringConsoleApp (Spring-Grundlagen) &rrar; WebApp (REST-Endpoints).
- **Tag 2:** WebApp (Fortsetzung) &rrar; Persistence Layer mit Spring Data, Domain Layer.
- **Tag 3:** WebApp (Fortsetzung) &rrar; Service-/Business-Layer, Fehlerbehandlung, Events, größere Übung (Schwein).

2 · Tag 1

Dependency Injection von Hand: Das CalculatorProject

2.1 Das Problem: feste Kopplung

Eine Klasse, die ihre Abhängigkeit selbst erzeugt, ist an eine konkrete Implementierung gekettet und nicht isoliert testbar:

Datei: CalculatorProject/src/client/CalcClient.java (Problemfall)

```
public class CalcClient {  
    private final CalculatorImpl calculator = new CalculatorImpl();  
    ...  
}
```

- Ein Austausch (z.B. gegen eine Version mit Logging) bedeutet: Code von calcClient ändern.
- CalcClient lässt sich nicht isoliert testen (kein Mock möglich).
- Verantwortung ist vermischt: CalcClient soll rechnen lassen, nicht entscheiden, WIE der Calculator gebaut wird.

2.2 Gegen eine Abstraktion programmieren

Ohne Interface gibt es nichts, was man „injizieren“ könnte – nur konkrete Klassen.

Datei: CalculatorProject/src/math/Calculator.java

```
public interface Calculator {  
    double add(double a, double b);  
    double sub(double a, double b);  
}
```

Datei: CalculatorProject/src/math/CalculatorImpl.java

```
public class CalculatorImpl implements Calculator {  
    @Override  
    public double add(double a, double b){ return a+b; }  
    @Override  
    public double sub(double a, double b){ return add(a, -b); }  
}
```

sub() ruft bewusst add() intern auf (this.add) – später, wenn Logging/Security als Wrapper darum gelegt werden, läuft genau dieser interne Aufruf NICHT durch den Wrapper. Ein typisches Proxy-/AOP-Problem, siehe Kapitel 13.

2.3 Konstruktor-Injektion

Datei: CalculatorProject/src/client/CalcClient.java

```
public class CalcClient {  
    private final Calculator calculator;  
  
    public CalcClient(final Calculator calculator) {  
        this.calculator = calculator;  
    }  
  
    public void go() {  
        System.out.println(calculator.add(1, 2));  
    }  
}
```

- CalcClient kennt nur das Interface Calculator, keine konkrete Klasse.

- Die Abhängigkeit wird von AUßEN über den Konstruktor hereingereicht, nicht selbst erzeugt.
- CalcClient entscheidet nicht, WELCHER Calculator verwendet wird – das entscheidet der Aufrufer.

Das ist bereits die von Spring empfohlene Form der Injektion: **Konstruktor-Injektion**. Wer „entscheidet“, welcher Calculator tatsächlich verwendet wird? Noch niemand – diese Verantwortung wandert immer weiter nach außen, bis sie am Ende beim Spring-Container landet.

2.4 Dekorierer: Abhängigkeiten kombinieren

Datei: CalculatorProject/src/math/CalculatorLogger.java

```
public class CalculatorLogger implements Calculator {
    private final Calculator calculator;

    public CalculatorLogger(Calculator calculator) {
        this.calculator = calculator;
    }

    @Override
    public double add(final double a, final double b) {
        System.out.println("Adding " + a + " " + b);
        return calculator.add(a, b);
    }

    @Override
    public double sub(final double a, final double b) {
        return calculator.sub(a, b);
    }
}
```

CalculatorSecure ist baugleich aufgebaut. Entscheidend: CalculatorLogger IST SELBST EIN Calculator UND BRAUCHT EINEN calculator im Konstruktor – damit lassen sich beliebig viele Varianten wie Matroschka-Puppen ineinanderstecken (**Decorator-Pattern**), ohne CalculatorImpl oder CalcClient anzufassen. Das funktioniert nur, weil konsequent gegen das Interface programmiert wurde.

Merksatz: DI ist die Voraussetzung dafür, dass sich Verhalten wie mit Bausteinen zusammenstecken lässt, ohne bestehenden Code zu ändern (Open/Closed-Prinzip).

2.5 Ausblick: der dynamische Proxy

Datei: CalculatorProject/src/common/LoggerProxy.java

```
public class LoggerProxy implements InvocationHandler {

    public static Object newInstance(Object obj) {
        return Proxy.newProxyInstance(
            obj.getClass().getClassLoader(),
            obj.getClass().getInterfaces(),
            new LoggerProxy(obj));
    }

    public Object invoke(Object proxy, Method m, Object[] args) throws Throwable {
        Object result;
        try {
            System.out.println("before method " + m.getName());
            result = m.invoke(obj, args);
        } catch (InvocationTargetException e) {
            throw e.getTargetException();
        } finally {
            System.out.println("after method " + m.getName());
        }
        return result;
    }
}
```

Statt für jedes Querschnittsthema eine eigene Decorator-Klasse zu schreiben, erzeugt hier die Java-Reflection-API zur Laufzeit ein Objekt, das Calculator implementiert, ohne dass wir eine eigene Klasse dafür schreiben müssen. Jeder Methodenaufruf läuft durch `invoke()`. Das ist noch kein Spring, aber genau das Prinzip, mit dem Spring AOP unter der Haube arbeitet (Kapitel 13) – und dasselbe Prinzip, mit dem Spring Data zur Laufzeit Repository-Implementierungen erzeugt (Kapitel 5).

2.6 Manuelle Verdrahtung in `main()`

```
Calculator calc = new CalculatorImpl();

//calc = new CalculatorLogger(calc);
calc = (Calculator) LoggerProxy.newInstance(calc);
calc = new CalculatorSecure(calc);
CalcClient calcClient = new CalcClient(calc);
calcClient.go();
```

`main()` ist der einzige Ort, der konkrete Klassen kennt und WEISS, welche Kombination von Decorators/Proxies verwendet werden soll. Genau diese Stelle – das manuelle „Zusammenstecken“ von Objektgraphen – nimmt uns später ein Framework ab: **Inversion of Control (IoC)**. Nicht mehr unser Code entscheidet aktiv (`new CalculatorImpl()`), sondern ein Container liest eine Konfiguration (Annotationen) und reicht uns fertige Objekte herein.

Merksatz für die Teilnehmer:

- **DI** = das Prinzip: Abhängigkeiten werden von außen hereingereicht.
- **IoC** = die Umkehrung der Kontrolle darüber, WER diese Objekte erzeugt und zusammensteckt.
- `Main.java` in diesem Beispiel IST unser Container – nur von Hand geschrieben, nicht automatisiert.

2.7 Zusammenfassung

1. DI beginnt mit einem Interface (`Calculator`) – ohne Abstraktion keine Austauschbarkeit.
2. Abhängigkeiten werden über den Konstruktor hereingereicht, nicht selbst erzeugt.
3. Weil alle Klassen gegen das Interface programmiert sind, lassen sich Verhalten wie Logging/Security beliebig kombinieren (Decorator-Pattern).
4. Ein dynamischer Proxy zeigt, wie sich Querschnittsthemen generisch lösen lassen – der Vorläufer von Spring AOP.
5. Irgendjemand muss am Ende trotzdem entscheiden und zusammenstecken; in diesem Beispiel macht das `main()` von Hand. Im nächsten Kapitel übernimmt das der Spring-Container automatisch.

3 · Tag 1

Spring-Grundlagen: Beans, Injektion, Lifecycle

3.1 Vom manuellen main() zum Spring-Container

```
Datei: SpringConsoleApp/.../SpringConsoleAppApplication.java
@SpringBootApplication
public class SpringConsoleAppApplication {
    public static void main(String[] args) {
        SpringApplication.run(SpringConsoleAppApplication.class, args);
    }
}
```

Verglichen mit Main.java aus Kapitel 2: Dort haben WIR jedes Objekt selbst erzeugt und verdrahtet. SpringApplication.run() startet stattdessen den **ApplicationContext** – den Spring-Container. Ab jetzt ist der Container das neue „Main.java“: Er durchsucht das Projekt, entscheidet, welche Objekte gebraucht werden, erzeugt sie und steckt sie zusammen – automatisch.

3.2 Was ist eine Bean? – @Component

```
Datei: SpringConsoleApp/.../demo/Demo.java
@Component
@Scope("singleton") // default
@RequiredArgsConstructor
public class Demo {

    private final Translator translator;

    @Value("${Demo.gruss}")
    private final String message;

    @PostConstruct
    public void init() {
        System.out.println(translator.translate("Post Construct von Demo"));
        System.out.println(message);
    }

    @PreDestroy
    public void peter() {
        System.out.println(translator.translate("Pre Destroy von Demo"));
    }
}
```

Eine **Bean** ist ein gewöhnliches Java-Objekt, das nicht vom eigenen Code, sondern vom Spring-Container erzeugt und verwaltet wird. @Component stellt eine Klasse unter genau diese Verwaltung – beim Start durchsucht Spring Boot automatisch das Package der @SpringBootApplication-Klasse und alle Unterpakete (**Component-Scan**).

Fachliche Spezialisierungen von @Component (technisch identisch, drücken aber die Absicht der Klasse aus): @Service (Business-Logik), @Repository (Datenzugriff), @Controller/@RestController (Web-Anfragen, siehe Kapitel 4).

3.3 Injektionsarten: Konstruktor, Setter/Field, und ihre Reihenfolge

Spring unterstützt grundsätzlich drei Wege, eine Abhängigkeit in eine Bean zu bekommen – in genau dieser Reihenfolge, wenn mehrere gleichzeitig vorkommen:

1. **Constructor-Injection** (von Spring empfohlen, siehe Demo oben) – Felder können final sein, das Objekt ist nach der Konstruktion vollständig und unveränderlich einsatzbereit.

2. **Setter- oder Field-Injection** – `@Autowired` auf einem Setter oder direkt auf dem Feld. Bequemer zu schreiben, aber das Feld kann nicht `final` sein und das Objekt existiert kurzzeitig unvollständig konstruiert.

3. `@PostConstruct` – läuft erst, nachdem alle Injektionen (egal welcher Art) abgeschlossen sind.

```
// Field-Injection (nur zur Illustration - im Kurs nicht verwendet)
@Component
public class DemoFieldInjected {
    @Autowired
    private Translator translator;
}

// Method-/Setter-Injection
@Component
public class DemoSetterInjected {
    private Translator translator;

    @Autowired
    public void setTranslator(Translator translator) {
        this.translator = translator;
    }
}
```

`@Autowired(required = false)` erlaubt es, eine Injektion optional zu machen – findet Spring keinen passenden Kandidaten, bleibt das Feld `null`/die Methode wird nicht aufgerufen, statt dass der Container-Start mit einem Fehler abbricht. Im Kurs bleibt Constructor-Injection der Standard: Sie macht Pflicht-Abhängigkeiten explizit und ermöglicht `final`-Felder.

3.4 Injektion mit Lombok: `@RequiredArgsConstructor`

Kein Konstruktor in Demo zu sehen – trotzdem wird `translator` injiziert. `@RequiredArgsConstructor` (Lombok) erzeugt zur Kompilierzeit automatisch genau den Konstruktor, den wir in Kapitel 2 bei `CalcClient` von Hand geschrieben haben. Spring sieht am Ende exakt denselben Bytecode wie bei einem handgeschriebenen Konstruktor.

Falle: `@Value("${Demo.gruss}")` steht in Demo auf dem FELD `message`. Lombok kopiert Feld-Annotationen NICHT automatisch auf den generierten Konstruktor-Parameter – und genau dort müsste `@Value` stehen, damit Spring den Wert einsetzen kann.

Datei: `lombok.config`

```
lombok.copyableAnnotations += org.springframework.beans.factory.annotation.Qualifier
lombok.copyableAnnotations += org.springframework.beans.factory.annotation.Value
```

Ohne diese Zeilen wäre `message` beim Start `null` bzw. der Start würde mit einem Injektionsfehler scheitern. **Lombok + Spring braucht an dieser Stelle eine bewusste Zusatzkonfiguration.**

3.5 `@Value`: Werte statt Objekte injizieren

Datei: `SpringConsoleApp/src/main/resources/application.properties`

```
spring.application.name=SpringConsoleApp
spring.profiles.active=production

Demo.gruss=Hallo Universum
```

`@Autowired` (implizit über den generierten Konstruktor) injiziert Objekte (Beans). `@Value("${Demo.gruss}")` injiziert stattdessen einen einfachen Wert aus der Konfiguration.

3.6 Bean-Lifecycle: `@PostConstruct` und `@PreDestroy`

`init()` (mit `@PostConstruct`) läuft automatisch, nachdem Demo konstruiert und `translator` injiziert wurde. `peter()` (mit `@PreDestroy`) läuft beim Herunterfahren des Containers.

Wichtig: `@PreDestroy` ist nur bei **Singleton**-Beans sinnvoll. Eine **Prototype**-Bean wird nach der Erzeugung an den Aufrufer „ausgehändigt“ und danach vom Container nicht weiter verwaltet – `@PreDestroy` würde dort nie aufgerufen.

3.7 Singleton, Prototype und Web-Scopes

```
@Scope("singleton") // default
//@Scope("prototype") // kein default
//@Lazy(true)
```

- `singleton` (Standard, auch ohne `@Scope`): der Container erzeugt genau **eine** Instanz.
- `prototype`: der Container erzeugt bei jeder Injektion/Anfrage eine **neue** Instanz und gibt danach die Kontrolle ab.
- `@Lazy(true)` verschiebt die Erzeugung einer Singleton-Bean vom Containerstart auf die erste tatsächliche Verwendung.

Für Web-Anwendungen gibt es zwei weitere Scopes, die nur dort Sinn ergeben: `@RequestScope` (eine Instanz pro HTTP-Request) und `@SessionScope` (eine Instanz pro HTTP-Session). Beide erzeugen intern einen Scoped Proxy, damit ein länger lebendes Objekt (z.B. ein Singleton-Controller) trotzdem eine pro Request/Session wechselnde Bean injiziert bekommen kann.

```
@Component
@RequestScope
public class RequestBezugeneKomponente { ... }
```

Zum Nachdenken: Zwischen `@Scope("singleton")` und `@Scope("prototype")` umschalten und beobachten, wie oft Konstruktor, `init()` und `postInit()` aufgerufen werden.

3.8 Wenn mehrere Kandidaten passen: Konflikte auflösen

```
public interface Translator {
    String translate(String input);
}

@Component
//@Qualifier("lower")
//@Primary
@Profile("test")
public class ToLowerTranslator implements Translator {
    public String translate(final String input) { return input.toLowerCase(); }
}

@Component
//@Qualifier("upper")
@Profile({"dev", "production"})
public class ToUpperTranslator implements Translator {
    public String translate(String input) { return input.toUpperCase(); }
}
```

Demo fragt nach einem `Translator`, es gibt aber zwei Implementierungen. Ohne weitere Hilfe sucht Spring **by Type**; gibt es mehrere Treffer, als fragilen Fallback **by Name**. Saubere Alternativen:

1. `@Primary` – bevorzugter Kandidat im Konfliktfall.
2. `@Qualifier("name")` – punktgenaue Auswahl über einen Namen.
3. `@Profile("...")` – die Bean wird nur registriert, wenn das Profil aktiv ist.

Aktiv ist hier `spring.profiles.active=production`: `ToUpperTranslator` trägt das Profil `production` und ist aktiv, `ToLowerTranslator` (Profil `test`) bleibt draußen – kein Konflikt.

3.9 Vom manuellen Main.java zur automatischen Verdrahtung

Dieselben vier Klassen aus Kapitel 2 – Calculator, CalculatorImpl, CalculatorLogger, CalculatorSecure – gibt es hier noch einmal, diesmal als Spring-Beans:

```
@Component
@Qualifier("impl")
public class CalculatorImpl implements Calculator {
    public double add(double a, double b){ return a+b; }
    public double sub(double a, double b){ return add(a, -b); }
}

@Component
@Qualifier("logger")
public class CalculatorLogger implements Calculator {
    private final Calculator calculator;
    public CalculatorLogger(@Qualifier("impl") Calculator calculator) {
        this.calculator = calculator;
    }
    ...
}

@Component
@Qualifier("secure")
public class CalculatorSecure implements Calculator {
    private final Calculator calculator;
    public CalculatorSecure(@Qualifier("logger") final Calculator calculator) {
        this.calculator = calculator;
    }
    ...
}
```

Jede Implementierung bekommt per `@Qualifier` einen eigenen Namen, jeder Konstruktor sagt per `@Qualifier` an seinem Parameter, welchen der drei Kandidaten er will. Der Container baut daraus beim Start automatisch dieselbe Kette wie in `Main.java` (Kapitel 2): `CalculatorSecure ⇉ CalculatorLogger ⇉ CalculatorImpl`.

Der zentrale Moment: `Main.java` (Kapitel 2) und die hier gezeigte `@Component/@Qualifier`-Verdrahtung führen zum selben Ergebnis. Der einzige Unterschied ist, WER die Verdrahtung übernimmt – wir von Hand oder der Container automatisch.

3.10 Zusammenfassung

1. **Bean** = ein von Spring verwaltetes Objekt; `@Component` + `Component-Scan` finden sie automatisch.
2. Drei Injektionsarten (Constructor/Setter/Field) mit fester Reihenfolge; Constructor-Injection bleibt der Standard.
3. **Lombok** erzeugt denselben Konstruktor, den man sonst von Hand schreibt – Vorsicht bei `@Value/@Qualifier` auf Feldern (`lombok.config`).
4. `@Value` injiziert Werte, nicht Objekte.
5. **Singleton/Prototype**/`@RequestScope`/`@SessionScope` und `@PostConstruct`/`@PreDestroy` steuern den Bean-Lifecycle.
6. Konflikte bei mehreren Kandidaten: `@Primary`, `@Qualifier`, `@Profile`.

4 · Tag 1

REST-Endpoints: der Presentation Layer

WebApp bleibt – anders als SpringConsoleApp – dauerhaft laufen und wartet auf HTTP-Anfragen; dafür bringt Spring Boot über spring-boot-starter-webmvc einen eingebetteten Server mit. Wir beginnen bewusst nur mit dem Presentation Layer.

4.1 Ressourcen-Naming

- Eine URL bezeichnet eine **Ressource** (ein Nomen), keine Aktion: /personen, nicht /getPersonen.
- Ressourcen-Sammlungen werden im **Plural** benannt: /personen liefert viele, /personen/{id} genau eine.
- Die **Aktion** wird über das **HTTP-Verb** transportiert (GET, POST, PUT, DELETE), nicht in die URL geschrieben.
- Eine sinnvolle **Versionierung** (hier /v1/...) macht die Schnittstelle stabil erweiterbar.

4.2 Minimalbeispiel: DemoController

Datei: WebApp/.../presentation/controller/DemoController.java

```
@RestController
@RequestMapping("/demo")
public class DemoController {

    @GetMapping(path = "/gruss", produces = MediaType.TEXT_PLAIN_VALUE)
    public String gruss() {
        return "Hallo Rest";
    }
}
```

@RestController ist eine spezialisierte @Component-Bean und sorgt zusätzlich dafür, dass Rückgabewerte direkt in den HTTP-Response-Body geschrieben werden. **Eine Methode + Annotationen = ein HTTP-Endpoint.**

4.3 Vollständiger Endpoint: PersonenController

Datei: WebApp/.../presentation/controller/PersonenController.java

```
@RestController
@RequestMapping("/v1/personen")
public class PersonenController {

    @GetMapping(path =("/{id}", produces = {MediaType.APPLICATION_JSON_VALUE,
    MediaType.APPLICATION_XML_VALUE})
    public ResponseEntity<PersonDto> getPerson(@PathVariable UUID id) {
        if (id.toString().endsWith("1")) {
            return ResponseEntity.notFound().build();
        }
        return
        ResponseEntity.ok(PersonDto.builder().id(id).vorname("Max").nachname("Mustermann").build());
    }

    @GetMapping(path = "", produces = MediaType.APPLICATION_JSON_VALUE)
    public ResponseEntity<Iterable<PersonDto>> getPersonen(
        @RequestParam(required = false, defaultValue = "Fritz") String vorname,
        @RequestParam(required = false, defaultValue = "Schmitt") String nachname
    ) {
        var list = List.of(
            PersonDto.builder().id(UUID.randomUUID()).vorname("John").nachname("Doe").build(
            ),

```

```

        PersonDto.builder().id(UUID.randomUUID()).vorname("John").nachname("Wick").build
    ()
    );
    return ResponseEntity.ok(list);
}

@DeleteMapping(path =("/{id}")
public ResponseEntity<Void> loeschePerson(@PathVariable UUID id) {
    if (id.toString().endsWith("1")) return ResponseEntity.notFound().build();
    return ResponseEntity.ok().build();
}

@PostMapping(path = "", consumes = MediaType.APPLICATION_JSON_VALUE)
public ResponseEntity<Void> einfuegen(@Valid @RequestBody PersonDto personDto,
UriComponentsBuilder uriBuilder) {
    UriComponents uriComponents =
uriBuilder.path("/v1/personen/{id}").buildAndExpand(personDto.getId());
    return ResponseEntity.created(uriComponents.toUri()).build();
}

@PutMapping(value =("/{id}", consumes = MediaType.APPLICATION_JSON_VALUE)
public ResponseEntity<Void> update(@PathVariable UUID id, @Valid @RequestBody PersonDto
personDto) {
    if (id.equals(personDto.getId())) throw new IdMismatchException("Upps");
    return ResponseEntity.ok().build();
}
}

```

Bewusster Unterschied zu einem „fertigen“ Controller: Dieser hat noch keine Abhängigkeit zu einem Service – er liefert sich selbst ausgedachte Daten und simuliert „nicht gefunden“, indem er prüft, ob die id mit „1“ endet. So lässt sich die REST-Mechanik isoliert besprechen, bevor Service und Persistence dazukommen (Kapitel 5–7).

- @PathVariable (Ressourcen-ID) vs. @RequestParam (Filter auf einer Sammlung, mit defaultValue).
- @Valid @RequestBody aktiviert die Bean-Validation aus PersonDto – ungültige Anfragen werden automatisch mit 400 Bad Request beantwortet.
- POST liefert bei Erfolg 201 Created mit Location-Header (UriComponentsBuilder).

Zum Nachdenken: Die Bedingung in update() wirft IdMismatchException, WENN die IDs übereinstimmen – fachlich würde man eher den gegenteiligen Fall erwarten. Lohnt sich, gemeinsam zu entdecken.

4.4 PersonDto

Datei: WebApp/.../presentation/dto/PersonDto.java

```

@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class PersonDto {

    @NotNull
    private UUID id;

    @NotNull
    @Size(min = 2, max = 50)
    private String vorname;

    @NotNull
    @Size(min = 2, max = 50)
    private String nachname;
}

```

@Data erzeugt Getter/Setter>equals/hashCode/toString, @Builder ermöglicht
PersonDto.builder()...build().

5 · Tag 2

Persistence Layer mit Spring Data JPA

Woher kommen die Daten, die PersonenController später über den Service liefert? Die Antwort liegt im Package persistence – Spring Data JPA setzt das DI-Prinzip aus Kapitel 2 fort: Wir schreiben ein Interface, Spring liefert zur Laufzeit die passende Implementierung als Bean.

5.1 Vom Java-Objekt zur Datenbanktabelle: @Entity

Datei: WebApp/.../persistence/entity/PersonEntity.java

```
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
@Entity
@Table(name = "tbl_personen")
public class PersonEntity {

    @Id
    @Column(nullable = false)
    private UUID id;

    @Column(nullable = false, length = 50)
    private String vorname;

    @Column(nullable = false, length = 50)
    private String nachname;
}
```

- @Entity stellt die Klasse unter die Verwaltung von JPA/Hibernate – eine Instanz entspricht ab jetzt einer Tabellenzeile.
- @Table(name = ...) legt die Zieltabelle fest; src/main/resources/data.sql befüllt sie beim Start.
- @Id markiert den Primärschlüssel – hier eine selbst vergebene UUID (kein @GeneratedValue).

PersonEntity kombiniert Lombok UND JPA-Annotationen auf derselben Klasse – kein Widerspruch: JPA/Hibernate braucht vor allem den parameterlosen Konstruktor (@NoArgsConstructor), um beim Auslesen per Reflection ein leeres Objekt zu erzeugen und zu befüllen.

PersonEntity sieht PersonDto sehr ähnlich – trotzdem zwei getrennte Klassen: Die eine beschreibt, WIE Daten über HTTP transportiert werden (inkl. Validierung), die andere, WIE sie in der Datenbank liegen (inkl. Spaltenlänge, Nullability).

5.2 Repository: Interface statt Implementierung

Datei: WebApp/.../persistence/repository/PersonenRepository.java

```
public interface PersonenRepository extends CrudRepository<PersonEntity, UUID> {
    Iterable<PersonEntity> findByVorname(String vorname);
}
```

PersonenRepository ist nur ein **Interface** – keine Klasse implementiert es. Trotzdem lässt es sich per Konstruktor injizieren und funktioniert. Spring Data erzeugt zur Laufzeit selbst eine Implementierung und registriert sie als Bean – technisch genau das Prinzip aus Kapitel 2.5 (LoggerProxy): ein dynamischer Proxy, der Methodenaufrufe abfängt und behandelt. CrudRepository bringt bereits Standard-Operationen mit: save(entity), findById(id) & Optional<T>, findAll(), existsById(id), deleteById(id).

5.3 Abgeleitete Query-Methoden

`findByVorname(String vorname)` steht nur als Signatur im Interface – Spring Data liest den **Methodennamen** und leitet daraus die Abfrage ab: `findBy + Vorname` wird zu `SELECT ... FROM tbl_personen WHERE vorname = ?`. Namenskonvention statt SQL/JPQL.

5.4 Zusammenfassung

1. `@Entity` bildet eine Java-Klasse auf eine Datenbanktabelle ab.
2. Ein Spring-Data-Repository ist nur ein **Interface** – die Implementierung erzeugt der Container zur Laufzeit automatisch (Proxy-Prinzip).
3. `CrudRepository` liefert Standard-Operationen ohne eine Zeile eigenen Codes.
4. Einfache eigene Abfragen entstehen über die **Methodennamen-Konvention**.

Vollständige Referenz: `<font face="Courier" size="9">docs.spring.io/spring-data/jpa/reference</font>`

6 · Tag 2

Domain Layer und Mapping

6.1 Person als fachliches Modell

Datei: WebApp/.../service/model/Person.java

```
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Person {
    private UUID id;
    private String vorname;
    private String nachname;
}
```

Person trägt keine JPA- und keine Bean-Validation-Annotationen – ein reines Java-Objekt ohne Framework-Abhängigkeit (nur Lombok gegen Boilerplate). Drei fast identisch aussehende Klassen (PersonDto, PersonEntity, Person) für dasselbe fachliche Ding, weil jede Schicht ihre eigene Sicht hat:

- PersonDto (Presentation) – wie sieht die Person „auf der Leitung“ (HTTP/JSON) aus, inkl. Validierungsregeln.
- PersonEntity (Persistence) – wie liegt die Person in der Datenbank.
- Person (Domain) – was IST eine Person fachlich, unabhängig von Transportweg oder Speicherung.

Der Service-Layer arbeitet ausschließlich mit Person – er kennt weder PersonDto noch PersonEntity. Die Umwandlung übernehmen eigene Mapper.

6.2 Mapper: MapStruct statt Hand-Code

Datei: WebApp/.../service/mapper/PersonMapper.java

```
@Mapper(componentModel = "spring")
public interface PersonMapper {
    Person convert(PersonEntity personEntity);
    PersonEntity convert(Person person);
    Iterable<Person> convert(Iterable<PersonEntity> personEntity);
}
```

Wieder nur ein Interface – keine Implementierung im Projekt, trotzdem per Konstruktor injizierbar. Unterschied zu Spring Data: Hier entsteht die Implementierung nicht zur Laufzeit per Proxy, sondern beim **Kompilieren** – mapstruct-processor generiert eine echte Klasse PersonMapperImpl mit lesbarem Java-Code, die als @Component registriert wird.

Merksatz: Zwei Frameworks, zwei Automatisierungs-Strategien, dasselbe Ziel: Spring Data erzeugt zur **Laufzeit** per dynamischem Proxy. MapStruct erzeugt zur **Kompilierzeit** per Annotation-Processor – kein Reflection-Overhead, dafür generierter Code bei jedem Build.

7 · Tag 3

Der Service-/Business-Layer

7.1 Vertrag und fachliche Prüfung

Datei: WebApp/.../service/PersonenService.java

```
public interface PersonenService {
    void speichern(Person person) throws PersonenServiceException;
    void aendern(Person person) throws PersonenServiceException;
    void loeschen(UUID uuid) throws PersonenServiceException;
    Optional<Person> findeNachId(UUID uuid) throws PersonenServiceException;
    Iterable<Person> findeAlle() throws PersonenServiceException;
}
```

Genau wie bei Calculator oder PersonenRepository gilt: Der Controller kennt nur dieses Interface, nicht PersonServiceImpl. Der Service übernimmt zwei Aufgaben, die weder Presentation noch Persistence übernehmen sollen: **Validierung** und eine **fachliche Prüfung**.

Datei: WebApp/.../service/internal/PersonServiceImpl.java Methode validieren

```
private void validieren(final Person person) throws PersonenServiceException {
    if (person == null) throw new PersonenServiceException("Person darf nicht null sein");
    if (person.getVorname() == null || person.getVorname().length() < 2) throw new
    PersonenServiceException("Vorname zu kurz");
    if (person.getNachname() == null || person.getNachname().length() < 2) throw new
    PersonenServiceException("Nachname zu kurz");
    if (antipathen.contains(person.getVorname())) throw new
    PersonenServiceException("Antipath");
}
```

Technische Mindestanforderungen (nicht null, Mindestlänge) und eine **fachliche** Regel: Steht der Vorname auf einer Liste unerwünschter Personen? Das Prinzip „fachliche Regeln gehören in den Service, nicht in Controller oder Persistence“ gilt unabhängig davon, ob man dafür eine eigene Exception-Klasse wählt.

7.2 Die Transaktionsklammer: @Transactional

```
@Service
@RequiredArgsConstructor
@Transactional(rollbackFor = PersonenServiceException.class, propagation =
Propagation.REQUIRED, isolation = Isolation.READ_COMMITTED)
public class PersonServiceImpl implements PersonenService {
    ...
}
```

@Transactional auf Klassenebene spannt um jede Methode eine Transaktionsklammer – Spring startet die Transaktion beim Methodenaufruf und committet (oder rollt zurück) beim Verlassen.

- propagation = Propagation.REQUIRED (Standard): existiert bereits eine laufende Transaktion, wird sie wiederverwendet; sonst wird eine neue eröffnet.
- isolation = Isolation.READ_COMMITTED: nur bereits committete Änderungen anderer Transaktionen sind sichtbar.
- rollbackFor = PersonenServiceException.class – der eigentliche Lehrpunkt: Spring rollt eine Transaktion standardmäßig nur bei **ungeprüften** Exceptions (RuntimeException) automatisch zurück. Bei **geprüften** (checked) Exceptions passiert das NICHT, außer man sagt es Spring explizit über rollbackFor.

PersonenServiceException ist deshalb bewusst eine **geprüfte** Exception (extends Exception) – sonst würde z.B. ein fehlgeschlagenes repo.save(...) innerhalb von speichern() NICHT zu einem Rollback führen, obwohl die Methode selbst eine PersonenServiceException wirft.

7.3 @Configuration und @Bean

Manche Abhängigkeiten (z.B. eine `List<String>` mit unerwünschten Vornamen) können nicht mit `@Component` markiert werden – man kann aus einer fremden Klasse keine eigene Komponente machen. Genau dafür gibt es `@Configuration/@Bean`:

```
Datei: WebApp/.../service/config/PersonConfig.java

@Configuration
public class PersonConfig {

    @Bean
    @Qualifier("antipathen")
    public List<String> createAntipathen() {
        return List.of("Attila", "Peter", "Paul", "Mary");
    }

    @Bean
    @Qualifier("fruits")
    public List<String> createFruits() {
        return List.of("Banana", "Cherry", "Strawberry", "Raspberry");
    }
}
```

`@Configuration` markiert eine Klasse als reine technische Quelle für Bean-Definitionen. Jede mit `@Bean` annotierte Methode ist eine kleine **Fabrik**: der Rückgabewert wird als Bean registriert. `@Qualifier` vergibt einen Namen, damit der Konstruktor von `PersonServiceImpl` genau die richtige `List<String>`-Bean bekommt und nicht eine andere.

7.4 @ConfigurationProperties: komplexe Werte aus externen Quellen

Für einfache Einzelwerte reicht `@Value`. Sobald mehrere zusammengehörige Einstellungen gebündelt und typsicher gebraucht werden (z.B. Mail-Server-Zugangsdaten), ist `@ConfigurationProperties` die passendere Wahl – und lässt sich sogar mit einer eigenen YAML-Datei kombinieren:

```
Datei: config/MailProperties.java

@Data
@ConfigurationProperties(prefix = "mail")
public class MailProperties {
    private String host;
    private int port;
    private String from;
}
```

```
Datei: config/YamlPropertySourceFactory.java

public class YamlPropertySourceFactory implements PropertySourceFactory {
    @Override
    public PropertySource<?> createPropertySource(String name, EncodedResource resource) {
        var factory = new YamlPropertiesFactoryBean();
        factory.setResources(resource.getResource());
        return new PropertiesPropertySource(resource.getResource().getFilename(),
            factory.getObject());
    }
}
```

```
Datei: config/MailConfig.java und src/main/resources/mail.yml

@Configuration
@EnableConfigurationProperties(MailProperties.class)
@PropertySource(value = "classpath:mail.yml", factory = YamlPropertySourceFactory.class)
public class MailConfig {
}

# mail.yml
mail:
  host: smtp.example.com
```

```
port: 587
from: noreply@example.com
```

`@ConfigurationProperties(prefix = "mail")` bindet alle Properties mit diesem Präfix typischer an die Felder von `MailProperties` – statt drei einzelner `@Value`-Injektionen gibt es ein einziges, testbares Objekt. `@PropertySource` liest standardmäßig nur `.properties`; für YAML braucht es die eigene factory (`YamlPropertySourceFactory`). `@EnableConfigurationProperties` registriert `MailProperties` selbst als Bean, obwohl die Klasse kein `@Component` trägt.

7.5 Ein Seitenblick auf SOLID

Konsequent zu Ende gedacht (Single Responsibility) gehört die Blacklist-Prüfung eigentlich nicht in `PersonServiceImpl`, sondern in einen eigenen Service – im Projekt bereits als Interface angelegt, aber didaktisch bewusst noch nicht verdrahtet:

```
public interface BlacklistService {
    boolean isBlacklisted(Person possibleBlacklistedPerson);
}
```

7.6 Unit-Testing mit Mockito

Datei: `WebApp/src/test/.../PersonServiceImplTest.java`

```
@ExtendWith(MockitoExtension.class)
class PersonServiceImplTest {
    @InjectMocks
    private PersonServiceImpl objectUnderTest;

    @Mock
    private PersonenRepository personRepositoryMock;
    @Mock
    private PersonMapper mapperMock;
    @Mock
    private List<String> antipathenMock;

    @Test
    void speichern__person_is_null__PersonenServiceExceptionIsThrown() {
        final PersonenServiceException ex = assertThrows(PersonenServiceException.class,
            () -> objectUnderTest.speichern(null));
        assertEquals("Person darf nicht null sein", ex.getMessage());
    }
}
```

Dieser Test startet **keinen** Spring-Container. `@Mock` erzeugt Test-Doubles, `@InjectMocks` reicht sie per Konstruktor in `objectUnderTest` – das funktioniert nur, weil `PersonServiceImpl` konsequent auf Konstruktor-Injektion gegen Interfaces setzt (Kapitel 2). Namenskonvention statt `@DisplayName: <Methode>__<Szenario>__<Erwartung>`.

8 · Tag 3

Fehlerbehandlung

8.1 Der zentrale ErrorHandler: @ControllerAdvice

```

Data:WebApp/.../presentation/errorhandler/ErrorHandler.java

@ControllerAdvice
public class ErrorHandler extends ResponseEntityExceptionHandler {

    @Override
    protected ResponseEntity<Object> handleMethodArgumentNotValid(
        final MethodArgumentNotValidException ex, final HttpHeaders headers,
        final HttpStatus code status, final WebRequest request) {

        Map<String, Object> body = new LinkedHashMap<>();
        body.put("timestamp", LocalDateTime.now());
        List<String> errors = ex.getBindingResult().getFieldErrors().stream()
            .map(x -> x.getField() + ":" + x.getDefaultMessage())
            .collect(Collectors.toList());
        body.put("errors", errors);

        logger.error("Upps", ex);
        return ResponseEntity.badRequest().body(body);
    }
    ...
}

```

@ControllerAdvice ist selbst nur eine Spezialisierung von @Component – mit einer Besonderheit: Sie gilt querschnittlich für alle @RestController-Beans. Genau wie LoggerProxy (Kapitel 2) ist das ein Werkzeug für Querschnittsthemen, diesmal für Exceptions. ResponseEntityExceptionHandler ist eine von Spring mitgelieferte Basisklasse mit fertigen Handlern für häufige Spring-MVC-Fälle; handleMethodArgumentNotValid feuert automatisch bei gescheiterter Bean-Validation.

8.2 Fachliche Exceptions auf Statuscodes abbilden

```

@ExceptionHandler(PersonenServiceException.class)
public ResponseEntity<Object> handlePersonenServiceException(PersonenServiceException ex,
    WebRequest request) {
    ...
    body.put("type", ex.getClass().getSimpleName()); // Achtung security
    logger.error("Upps", ex);
    return ResponseEntity.internalServerError().body(body);
}

@ExceptionHandler(AlreadyExistsException.class)
public ResponseEntity<Object> handleAlreadyExistsException(AlreadyExistsException ex,
    WebRequest request) {
    logger.error("Upps", ex);
    return ResponseEntity.status(HttpStatus.CONFLICT).body(ex.getMessage());
}

@ExceptionHandler(NotFoundException.class)
public ResponseEntity<Object> handleNotFoundException(NotFoundException ex, WebRequest
    request) {
    logger.error("Upps", ex);
    return ResponseEntity.notFound().build();
}

@ExceptionHandler(IdMismatchException.class)
public ResponseEntity<Object> handleIdMismatchException(IdMismatchException ex, WebRequest
    request) {
    logger.error("Upps", ex);
}

```

```
    return ResponseEntity.badRequest().body(ex.getMessage());  
}
```

Ein `@ExceptionHandler(XyzException.class)` pro Exception-Typ, jeder mit einer bewussten Statuscode-Entscheidung. Für `@ExceptionHandler` spielt es keine Rolle, ob eine Exception checked oder unchecked ist – es zählt nur der Typ (ein Kontrast zu 7.2, wo das bei `rollbackFor` sehr wohl entscheidend war).

Security-Hinweis: Den internen Exception-Typ (oder gar `ex.getMessage()` mit technischen Details) ungefiltert an den Client zurückzugeben, kann Architektur preisgeben (Information Disclosure). Serverseitig wird trotzdem IMMER geloggt – intern alles protokollieren, nach außen nur das Nötigste zeigen.

Zum Nachdenken: Ein generischer `Exception.class`-Handler als letztes Sicherheitsnetz fehlt hier bislang. Was passiert bei einer unerwarteten `NullPointerException`, die keiner dieser Klassen entspricht? Sollte ein Auffang-Handler ergänzt werden, der niemals einen nackten Stacktrace durchlässt?

9 · Tag 3

Größere Übung: alle Schichten selbst bauen (Schwein)

Jetzt, wo Persistence, Domain, Service und Fehlerbehandlung komplett an Person durchgespielt sind, wird alles selbst angewendet: Persistence, Service und REST-Endpoint für eine neue Ressource – Schwein – komplett selbst bauen, inklusive der Mapper dazwischen.

9.1 Aufgabenstellung

Denselben Dreiklang wie bei Person bauen, in der Reihenfolge, in der man eine Ressource in der Praxis typischerweise **baut**:

1. **Persistence Layer**: SchweinEntity (@Entity, @Id, @Column) + SchweinRepository (CrudRepository<SchweinEntity, UUID>).
2. **Mapper** zwischen Entity und Domain-Modell (MapStruct).
3. **Domain Layer**: Schwein – mit Zusatzanforderung, siehe unten.
4. **Service Layer**: SchweineService/SchweineServiceImpl – CRUD plus fachliche Aktion **Füttern**, die das Gewicht erhöht.
5. **Mapper** zwischen Domain-Modell und DTO.
6. **Presentation Layer**: SchweinDto + SchweinController – CRUD nach denselben Ressourcen-Naming-Regeln, plus ein Endpoint fürs Füttern.
7. **Fehlerbehandlung erweitern**: SchweineServiceException braucht einen eigenen @ExceptionHandler.

Wichtige Zusatzanforderung: Person war bisher ein reines **Anemic Domain Model** (Martin Fowler) – nur Felder und Getter, jede Regel steckte im Service. Schwein soll das NICHT sein: Die Regel „Füttern erhöht das Gewicht“ gehört auf das Domain-Objekt selbst (schwein.fuettern()), nicht als Rechnung im Service. Der Service orchestriert nur noch (laden, Methode aufrufen, speichern).

Zum Füttern-Endpoint: „Füttern“ ist keine der vier Standard-Operationen auf /v1/schweine/{id} – ein Endpoint wie /v1/schweine/{id}/fuettern wäre ein Verb in der URL und verstößt gegen die Regel „Ressourcen sind Nomen“. Die gängige Alternative: die Aktion selbst als Unterressource modellieren – eine Fütterung ist ein Ereignis, das man **anlegt**: POST /v1/schweine/{id}/fuetterungen.

9.2 In welcher Reihenfolge bauen?

- **Bottom-up, von der Persistence aus:** frühzeitig gegen eine echte Datenbank testbar; Risiko, dass sich das API-Design zu sehr an der Datenbankstruktur orientiert.
- **Top-down/outside-in, vom Controller aus** (Contract-first): API-Design bleibt am Bedarf orientiert; ohne Stubs zwischenzeitlich nicht lauffähig.
- **Middle-out, vom Domain-Modell aus:** fachlicher Kern zuerst, ganz ohne Spring/JPA/HTTP, isoliert testbar; Persistence und Presentation werden als „Adapter“ drumherum gebaut (Kerngedanke der Onion-/Hexagonal-Architektur).

9.3 Musterlösung

Schicht	Datei
Entity	persistence/entity/SchweinEntity.java
Repository	persistence/repository/SchweinRepository.java

Schicht	Datei
Mapper (Entity ↔ Domain)	service/mapper/SchweinMapper.java
Domain-Modell	service/model/Schwein.java
Service-Interface	service/SchweineService.java
Service-Implementierung	service/internal/SchweineServiceImpl.java
Mapper (Domain ↔ DTO)	presentation/mapper/SchweinDtoMapper.java
DTO	presentation/dto/SchweinDto.java
Controller	presentation/controller/v1/SchweinController.java
Fehlerbehandlung	presentation/errorhandler/ErrorHandler.java

Der entscheidende Ausschnitt – das nicht-anämische Domain-Modell:

Datei: service/model/Schwein.java

```

@Data
@Setter(AccessLevel.PRIVATE)
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Schwein {

    private UUID id;
    private String name;
    private int gewicht;

    public void fuettern() {
        setGewicht(getGewicht() + 1);
    }
}

```

`@Setter(AccessLevel.PRIVATE)` überschreibt gezielt, was `@Data` normalerweise erzeugen würde (öffentliche Setter). Von außen – auch vom Service! – lässt sich gewicht dadurch NICHT mehr direkt setzen, nur über `fuettern()`. Das ist Kapselung im eigentlichen Sinne und macht ein nicht-anämisches Domain-Modell erst wirklich wasserdicht.

Datei: service/internal/SchweineServiceImpl.java

```

@Override
public void fuettern(final UUID uuid) {
    try {
        Schwein schwein = repo.findById(uuid)
            .map(mapper::convert)
            .orElseThrow(() -> new NotFoundException("Schwein konnte nicht gefunden
werden"));
        schwein.fuettern();
        repo.save(mapper.convert(schwein));
    } catch (NotFoundException e) {
        throw e;
    } catch (RuntimeException e) {
        throw new SchweineServiceException("Fehler beim Fuettern", e);
    }
}

```

Der Service lädt, ruft die fachliche Methode auf dem Domain-Objekt auf und speichert – die Berechnung selbst steht ausschließlich in `Schwein.fuettern()`. Das ist der Unterschied zwischen anämisch und nicht-anämisch in einem Satz.

Zwei weitere Vergleichspunkte zu Person: `SchweineServiceException` ist bewusst **ungeprüft** (extends `RuntimeException`) – deshalb reicht `SchweineServiceImpl` ein blankes `@Transactional`

(Standard-Rollback bei jeder `RuntimeException` greift automatisch), während `PersonServiceImpl` explizit `rollbackFor` braucht (7.2).

10 · Tag 3

Integrationstests gegen den laufenden Container

10.1 Vom Unit-Test zum echten HTTP-Test

```

DataJdbcWebApp/src/test/.../PersonenControllerTest.java

@ActiveProfiles("test")
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
@AutoConfigureTestRestTemplate
@ExtendWith(SpringExtension.class)
@Sql({"/create.sql", "/insert.sql"})
class PersonenControllerTest {

    @Autowired
    private TestRestTemplate restTemplate;

    @MockitoBean
    private PersonenService personenServiceMock;

    @Test
    void findByTest() throws PersonenServiceException {
        final var optionalPerson = Optional.of(new Person(
            UUID.fromString("b2e24e74-8686-43ea-baff-d9396b4202e0"), "John", "Doe"));
        when(personenServiceMock.findeNachId(any())).thenReturn(optionalPerson);

        var personDto = restTemplate.getForObject(
            "/v1/personen/b2e24e74-8686-43ea-baff-d9396b4202e0", PersonDto.class);
        assertEquals("John", personDto.getVorname());
    }

    @Test
    void test4() throws PersonenServiceException {
        when(personenServiceMock.findeNachId(any())).thenReturn(Optional.empty());
        var entity = restTemplate.getForEntity(
            "/v1/personen/b2e24e74-8686-43ea-baff-d9396b4202e0", PersonDto.class);
        assertEquals(HttpStatus.NOT_FOUND, entity.getStatusCode());
    }
}

```

Im Unterschied zu 7.6 (reiner Mockito-Unit-Test, kein Spring involviert) startet `@SpringBootTest(webEnvironment = RANDOM_PORT)` den **vollständigen** Spring-Container inklusive eingebettetem Server auf einem freien Port – echtes HTTP, echtes MVC-Routing, echte JSON-(De-)Serialisierung.

- `TestRestTemplate` ist ein von Spring Boot bereitgestellter HTTP-Client für Tests. In Spring Boot 4.1 braucht es dazu zusätzlich `@AutoConfigureTestRestTemplate`.
- `@MockitoBean` ersetzt eine **echte Bean im laufenden ApplicationContext** durch ein Mockito-Mock – der Controller ist echt, nur der Service dahinter wird ausgetauscht.
- `@ActiveProfiles("test")` aktiviert eine isolierte Test-Konfiguration mit H2-In-Memory-Datenbank; `@Sql({...})` lädt Testdaten.
- `test4` zeigt: `ResponseEntity.of(Optional)` liefert bei leerem `Optional` automatisch 404 Not Found – hier end-to-end über einen echten HTTP-Request nachgewiesen.

Zwei Testebenen, zwei Fragen: Der Mockito-Unit-Test beantwortet „Stimmt meine Business-Logik?“ (ohne Spring, sehr schnell). Der Integrationstest beantwortet „Kommt bei einem echten HTTP-Request wirklich das richtige JSON mit dem richtigen Statuscode heraus?“ (mit Spring und echtem Server, dafür langsamer). Beide Ebenen ergänzen sich.

11 · Tag 3

Lose Kopplung durch Events

Bisher hat jede Schicht ihre Nachbarschicht direkt gerufen: Controller ruft Service, Service ruft Repository. Ein Event dreht dieses Prinzip um: Der Service **meldet** nur, dass etwas passiert ist – ohne zu wissen (oder wissen zu wollen), wer diese Meldung interessiert.

11.1 ApplicationEventPublisher

Datei: WebApp/.../event/PersonCreatedEvent.java

```
public record PersonCreatedEvent(UUID id, String vorname, String nachname) {
}
```

Ein Event ist hier nur ein einfaches, unveränderliches Datenobjekt (record) – seit Spring 4.2 akzeptiert `ApplicationEventPublisher.publishEvent(...)` ein beliebiges POJO/record.

Datei: WebApp/.../service/internal/PersonServiceImpl.java

```
private final ApplicationEventPublisher applicationEventPublisher;

@Override
public void speichern(Person person) throws PersonenServiceException {
    try {
        validieren(person);
        if (repo.existsById(person.getId())) throw new AlreadyExistsException("Datensatz existiert bereits");
        repo.save(mapper.convert(person));
        applicationEventPublisher.publishEvent(
            new PersonCreatedEvent(person.getId(), person.getVorname(),
            person.getNachname()));
    } catch (AlreadyExistsException e) {
        throw e;
    } catch (RuntimeException e) {
        throw new PersonenServiceException("Fehler beim Speichern", e);
    }
}
```

`ApplicationEventPublisher` ist eine von Spring bereitgestellte Schnittstelle – der `ApplicationContext` selbst implementiert sie. `publishEvent(...)` wird erst nach erfolgreichem `repo.save(...)` aufgerufen – schlägt das Speichern fehl, wird kein Event ausgelöst.

Datei: WebApp/.../MyEventListener.java

```
@Component
public class MyEventListener {
    @EventListener
    public void handlePersonCreatedEvent(PersonCreatedEvent event) {
        System.out.println("PersonCreatedEvent wurde ausgelöst: " + event);
    }
}
```

`@EventListener` markiert eine Methode als Empfänger – Spring entscheidet allein anhand des **Parametertyps**, welche Methode bei welchem Event aufgerufen wird.

Merksatz: Der Service kennt den Listener nicht, der Listener kennt den Service nicht – beide kennen nur das Event. Das Observer-Pattern, von Spring als Publish/Subscribe-Mechanismus fertig bereitgestellt. Beliebige weitere Listener lassen sich ergänzen, ohne den Service anzufassen.

Zum Nachdenken: `@EventListener` läuft standardmäßig synchron und in derselben Transaktion wie der Publisher. Was passiert, wenn der Listener selbst eine `RuntimeException` wirft? Und welchen Unterschied macht `@TransactionalEventListener(phase = AFTER_COMMIT)`?

12 · Tag 3

Fortgeschrittene Repository-Strategien

12.1 @Query, Projektionen und Custom-Repositories

CrudRepository und die Methodennamen-Konvention (Kapitel 5) sind zwei von drei Wegen, wie ein Repository zu seinen Abfragen kommt. PersonenRepository zeigt inzwischen alle drei Strategien gleichzeitig:

```
public interface PersonenRepository extends CrudRepository<PersonEntity, UUID>,
    PersonenCustomRepository {

    Iterable<PersonEntity> findByVorname(String vorname);

    @Query("select new de.fi.webapp.persistence.entity.TinyPerson(p.id, p.nachname) from
    PersonEntity p")
    Iterable<TinyPerson> egal();

    Iterable<TinyPerson> findAllProjectByVorname(String vorname);
}
```

- egal() nutzt eigenes JPQL mit einer **Constructor-Expression**: Statt ganzer PersonEntity-Objekte baut JPA pro Zeile direkt ein TinyPerson (ein record mit nur id/nachname) – nützlich, wenn nicht jede Spalte gebraucht wird.
- findAllProjectByVorname(...) kombiniert dieselbe Projektions-Idee mit der Methodennamen-Konvention: Spring Data erkennt am Rückgabety, dass eine DTO-Projektion gewünscht ist.

Datei: persistence/repository/PersonenCustomRepository(Impl).java

```
public interface PersonenCustomRepository {
    void onlySave(PersonEntity personEntity);
}

public class PersonenCustomRepositoryImpl implements PersonenCustomRepository {

    @PersistenceContext
    private EntityManager em;

    @Override
    @Transactional(propagation = Propagation.REQUIRES_NEW)
    public void onlySave(final PersonEntity personEntity) {
        try {
            em.persist(personEntity);
            em.flush();
        } catch (EntityExistsException | DataIntegrityViolationException e) {
            throw new AlreadyExistsException(e.getMessage());
        }
    }
}
```

Manchmal reicht selbst @Query nicht – die **dritte** Strategie ist eine Custom-Repository-Implementierung mit voller Kontrolle über die JPA-API. @PersistenceContext injiziert den EntityManager über die **JPA-Standard-Annotation** (funktioniert auch außerhalb von Spring). em.persist() + em.flush() erzwingt, dass der INSERT sofort an die Datenbank geschickt wird – anders als save(), das Hibernate normalerweise erst am Ende der Transaktion ausführt. Propagation.REQUIRES_NEW startet bewusst eine eigene, neue Transaktion, unabhängig von einer eventuell schon laufenden.

PersonenRepository extends CrudRepository<...>, PersonenCustomRepository erbt gleichzeitig von Strategie 1 (automatischer Proxy) und Strategie 3 (Handschrift). Spring Data erkennt allein an der Namenskonvention (PersonenCustomRepository + Suffix Impl), dass Aufrufe an die handgeschriebene

Klasse delegiert werden sollen – keine explizite Verdrahtung nötig.

13 · Vertiefung

Aspektorientierte Programmierung (AOP) mit Spring

Kapitel 2 hat mit `LoggerProxy` einen dynamischen Proxy von Hand gebaut, um Logging als Querschnittsthema um `Calculator` zu legen – ohne `CalculatorImpl` anzufassen. Genau dieses Prinzip industrialisiert Spring unter dem Namen **AOP (Aspect Oriented Programming)**: Statt für jedes Querschnittsthema eine eigene Decorator- oder Proxy-Klasse zu schreiben, beschreibt man deklarativ per Annotation, WELCHE Methoden betroffen sind und WAS davor/danach passieren soll – Spring erzeugt den Proxy dahinter automatisch.

Voraussetzung ist die Dependency `spring-boot-starter-aop` in der `pom.xml`.

13.1 @Aspect und @Pointcut

Datei: `aspect/LoggingAspect.java`

```
@Aspect
@Component
public class LoggingAspect {

    @Pointcut("execution(* de.fi.webapp.service..*ServiceImpl.*(..))")
    public void serviceLayer() {}
}
```

`@Aspect` markiert eine Klasse als Aspekt, `@Component` macht sie zur Bean, damit Spring sie überhaupt findet. `@Pointcut` beschreibt über einen **execution-Ausdruck**, WELCHE Methodenaufrufe betroffen sind – hier: alle öffentlichen Methoden jeder Klasse, deren Name auf `ServiceImpl` endet, im Package `service` und dessen Unterpaketen (`..`).

13.2 Before-/After-Advice

```
@Aspect
@Component
@RequiredArgsConstructor
public class LoggingAspect {

    private static final Logger log = LoggerFactory.getLogger(LoggingAspect.class);

    @Pointcut("execution(* de.fi.webapp.service..*ServiceImpl.*(..))")
    public void serviceLayer() {}

    @Before("serviceLayer()")
    public void logBefore(JoinPoint jp) {
        log.info("Aufruf von {} mit Argumenten {}", jp.getSignature().getName(),
            jp.getArgs());
    }

    @AfterReturning(pointcut = "serviceLayer()", returning = "result")
    public void logAfterReturning(JoinPoint jp, Object result) {
        log.info("{} erfolgreich beendet, Ergebnis: {}", jp.getSignature().getName(),
            result);
    }

    @AfterThrowing(pointcut = "serviceLayer()", throwing = "ex")
    public void logAfterThrowing(JoinPoint jp, Exception ex) {
        log.error("{} mit Exception beendet: {}", jp.getSignature().getName(),
            ex.getMessage());
    }
}
```

- `@Before` läuft, bevor die eigentliche Methode ausgeführt wird – kann den Aufruf aber nicht verhindern.
- `@AfterReturning` läuft nur bei erfolgreicher Rückkehr; über `returning` steht der Rückgabewert zur Verfügung.
- `@AfterThrowing` läuft nur, wenn eine Exception den Methodenaufruf verlassen hat.
- `@After` (nicht abgebildet) läuft immer – wie ein `finally`-Block, egal ob Erfolg oder Exception.

13.3 @Around: volle Kontrolle

```
@Around("serviceLayer()")
public Object logAround(ProceedingJoinPoint pjp) throws Throwable {
    long start = System.currentTimeMillis();
    try {
        return pjp.proceed();
    } finally {
        log.info("{} dauerte {} ms", pjp.getSignature().getName(), System.currentTimeMillis()
- start);
    }
}
```

`@Around` ist der mächtigste Advice-Typ: `ProceedingJoinPoint.proceed()` entspricht genau dem `m.invoke(obj, args)` aus `LoggerProxy` (Kapitel 2.5) – der Aspekt entscheidet sogar, OB die eigentliche Methode überhaupt aufgerufen wird, und kann das Ergebnis vor der Rückgabe noch verändern.

13.4 Eigene Annotation als Pointcut

Statt einen Pointcut über Package-/Klassennamen zu beschreiben (fragil bei Umbenennungen), lässt er sich auch gezielt über eine selbst definierte Annotation ansprechen:

```
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface Loggable {
}

@Around("@annotation(Loggable)")
public Object logAnnotated(ProceedingJoinPoint pjp) throws Throwable {
    log.info("Aufruf von {}", pjp.getSignature().getName());
    return pjp.proceed();
}

// Verwendung:
@Loggable
public void speichern(Person person) throws PersonenServiceException { ... }
```

`@annotation(Loggable)` greift nur bei Methoden, die explizit mit `@Loggable` markiert sind – unabhängig davon, in welchem Package oder in welcher Klasse sie liegen.

13.5 Vergleich zu Kapitel 2

Ansatz	Wo aus Kapitel 2/12	Wann sinnvoll
Decorator	CalculatorLogger, CalculatorSecure	Wenige, feste Kombinationen; Verhalten gehört sichtbar zum Objektgraphen.
Dynamischer Proxy (Handschrift)	LoggerProxy	Ein Querschnittsthema, keine Spring-Abhängigkeit gewünscht.
Spring AOP (@Aspect)	Dieses Kapitel	Viele Methoden/Klassen betroffen, deklarativ über Pointcuts statt manuellem Verdrahten.

Merksatz: Spring Data (Kapitel 5) und Spring AOP nutzen technisch denselben Trick: einen zur Laufzeit erzeugten Proxy, der Methodenaufrufe abfängt. Der Unterschied ist nur, WOFÜR er benutzt wird – fertige

CRUD-Implementierungen oder Querschnittslogik.

14

Ausblick: was noch fehlt

Bewusst zurückgestellt, Themen für einen der nächsten Termine:

- **@Order** zur Steuerung der Ausführungsreihenfolge mehrerer CommandLineRunner-Beans, sobald mehr als einer aktiv ist.
- Ein reines Lombok-Beispiel (@Data, @NoArgsConstructor, @AllArgsConstructor, @Builder) losgelöst von einer konkreten fachlichen Klasse, als zusätzliche Übung zum Lombok-Muster aus Kapitel 4.4.
- **Swagger/OpenAPI im Detail** (@Operation/@ApiResponses) – springdoc-openapi ist bereits als Dependency eingebunden (siehe WebApp/DEPENDENCIES.md), die Feinsteuerung der generierten Dokumentation wurde bisher nur am Rande erwähnt.
- **AspectJ-Weaving zur Kompilierzeit** als Alternative zum laufzeitbasierten Proxy aus Kapitel 13 – relevant, sobald auch private oder finale Methoden bzw. interne Selbstaufrufe (this.methode(), vgl. Kapitel 2.2) von einem Aspekt erfasst werden müssen.

Dieses Skript ersetzt für diesen Kurs die ältere Broschüre über klassisches Spring (XML-Konfiguration, ClassPathXmlApplicationContext, JdbcTemplate, RMI/JMS/MBeans) – diese Themen sind in einem modernen Spring-Boot-Seminar mit Spring Data JPA und REST nicht mehr der praxisrelevante Einstieg und wurden hier bewusst nicht übernommen.